Name:

PRN:

Experiment no 8

Title: Comparative Study of OpenMP and MPI for Parallel Applications

1. **Aim:**

To implement selected computationally intensive applications using both OpenMP and MPI, and experimentally compare their performance, scalability, memory model, communication overhead, and suitability for different HPC workloads.

You will:

- Implement the same computational problem using OpenMP and MPI.
- Apply shared-memory and distributed-memory parallel programming models.
- Compare execution time, speedup, efficiency and scalability.
- Analyse the effect of increasing threads/processes.
- Identify the advantages and limitations of OpenMP and MPI for different applications.
- Relate the experimental observations to real-world HPC and AI workloads.

2. **Why?**

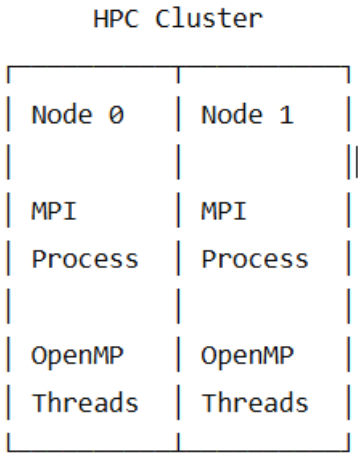
OpenMP and MPI are two fundamental programming models used in HPC, but they solve parallel programming problems differently. The same problem can often be solved using both approaches, but their performance can be different. Therefore, an HPC programmer should not only know how to parallelize a program, but also understand: Which parallel programming model is appropriate for a given problem and architecture?

3. **HPC in Practice**

Modern HPC applications frequently use combinations of programming models.

For example, a cluster node may contain multiple CPU cores.

A distributed application can use:



This is known as hybrid MPI + OpenMP programming.

Applications such as:

- Weather and climate simulation
- Computational fluid dynamics
- Molecular dynamics
- Image processing
- Deep learning
- Scientific simulations

can use these programming models.

4. **Learning Focus**

Concept	**OpenMP**	**MPI**
Memory model	Shared memory	Distributed memory
Parallel unit	Thread	Process
Communication	Shared variables	Message passing
Data sharing	Implicit/shared	Explicit
Scalability	Within shared-memory system	Across multiple nodes
Synchronization	Critical, barrier, atomic, reduction	Send/Receive, collective communication

Programming complexity	Generally lower	Generally higher
Multi-node execution	Limited	Well suited

5. Pre-Lab Activity

Before starting the implementation, answer:

1. What is the difference between a thread and an MPI process?
2. What is shared-memory parallelism?
3. What is distributed-memory parallelism?
4. How does OpenMP distribute loop iterations?
5. How does MPI distribute data among processes?
6. What is communication overhead?
7. Why can an MPI program be slower than OpenMP for some problems?
8. Why is MPI more suitable for multi-node systems?
9. What is strong scaling?
10. What is weak scaling?

Section A – Task 1: Matrix–Matrix Multiplication

Problem Statement

Matrix multiplication is selected because it is computationally intensive and provides sufficient workload for meaningful performance comparison.

Task 1A – Sequential Implementation

Suggested sizes:

- 500 × 500
- 1000 × 1000
- 2000 × 2000

Task 1B – OpenMP Implementation

Parallelize the matrix multiplication using OpenMP.

Use:

#pragma omp parallel for

Investigate different numbers of threads:

- 2
- 4
- 8
- Maximum available threads

Measure execution time.

Task 1C – MPI Implementation

Implement the same matrix multiplication using MPI.

Use **row-wise decomposition**.

Suggested strategy:

1. Process 0 initializes matrices (A) and (B).
2. Distribute rows of (A) among MPI processes.
3. Broadcast matrix (B).
4. Each process computes its assigned rows.
5. Gather the resulting rows of (C).

Use appropriate MPI collective operations.

Task 1D – Compare OpenMP and MPI

For the same matrix sizes, compare:

- Sequential
- OpenMP
- MPI

Prepare:

Matrix Size	Parallel Model	Threads/Processes	Execution Time	Speedup	Efficiency
500	Sequential	1			
500	OpenMP	2			
500	OpenMP	4			
500	OpenMP	8			

500	MPI	2			
500	MPI	4			
500	MPI	8			

Repeat for larger matrix sizes.

Analyze

1. Which implementation has lower execution time?
2. How does performance change as threads/processes increase?
3. What is the communication overhead in MPI?
4. Why does OpenMP have lower communication overhead on a shared-memory system?
5. At what problem size does parallel execution provide significant benefit?

Section B – Task 2: 2D Convolution

Problem Statement

Implement a 2D convolution operation on a large matrix representing an image or scientific data grid.

Given an input matrix (I) and convolution kernel (K), calculate:

[
O(i,j)=\sum_{u}\sum_{v} I(i+u,j+v)K(u,v)
]

Use a small kernel such as:

1 1 1

1 1 1

1 1 1

or a suitable edge-detection kernel.

Task 2A – Sequential Implementation

Implement the 2D convolution sequentially.

Measure execution time for large input sizes.

Suggested sizes:

- 1000 × 1000
- 2000 × 2000
- 4000 × 4000

Task 2B – OpenMP Implementation

Parallelize the convolution using OpenMP.

Investigate:

- Parallelizing the outer loop.
- Number of threads.
- Scheduling strategy.

Compare:

static

dynamic

guided

where appropriate.

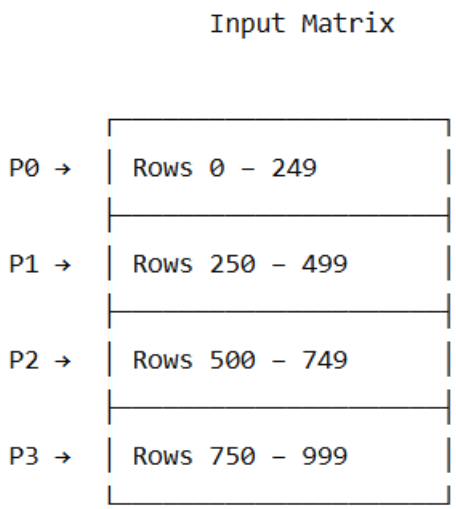
Measure execution time and analyze the effect of scheduling.

Task 2C – MPI Implementation

Implement 2D convolution using MPI.

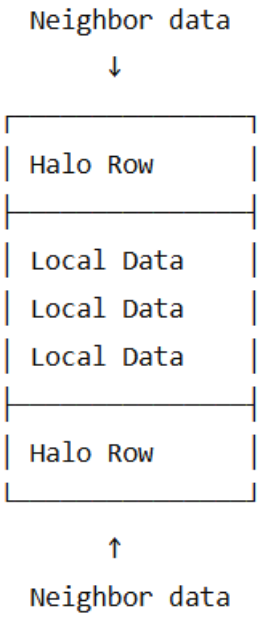
Use **row-wise domain decomposition**.

For example:



Because convolution requires neighboring rows, investigate the concept of boundary/halo data.

Each process may need additional rows from neighboring processes.



Investigate how MPI communication can be used to exchange boundary information.

Section C – Strong Comparative Analysis

Task 3: OpenMP vs MPI Performance Comparison

For both applications:

Application 1

Matrix–Matrix Multiplication

Application 2

2D Convolution

Compare:

- Sequential execution
- OpenMP execution
- MPI execution

using equivalent computational workloads.

6. Performance Metrics

Calculate: **Execution Time**, **Speedup**, **Efficiency**

7. Required Experiments

For each application, perform experiments using:

Sequential

1 execution

OpenMP

2 threads

4 threads

8 threads

Maximum available threads

MPI

2 processes

4 processes

8 processes

Use the **same input data and computational problem** for all implementations.

8. Performance Graphs

Prepare the following graphs for both applications:

Graph 1

Number of Threads/Processes vs Execution Time

Compare:

OpenMP

MPI

Graph 2

Number of Threads/Processes vs Speedup

Graph 3

Number of Threads/Processes vs Efficiency

Graph 4

Problem Size vs Execution Time

Compare:

Sequential

OpenMP

MPI

9. Architecture and Memory Analysis

Prepare a conceptual comparison:

Feature	OpenMP	MPI
Memory model		
Execution unit		
Communication mechanism		
Synchronization		
Data distribution		
Communication overhead		
Scalability		
Multi-node suitability		
Programming complexity		

Then answer:

1. Why does OpenMP perform well on a shared-memory multicore system?
2. Why does MPI introduce communication overhead?
3. Why can MPI scale across multiple nodes?
4. Why may OpenMP not be sufficient for very large cluster-based problems?
5. How does data decomposition differ between OpenMP and MPI?

10. Challenge Task – Hybrid Thinking

Consider an HPC cluster containing:

4 Compute Nodes

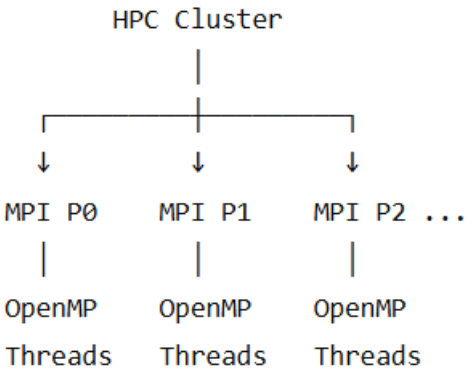
↓

8 CPU Cores per Node

Design a conceptual **hybrid MPI + OpenMP solution** for matrix multiplication.

For example:

HPC Cluster



Explain:

1. How MPI processes would be distributed across nodes.
2. How OpenMP threads would operate within each process.
3. How matrix data would be distributed.
4. Why hybrid programming may reduce the number of MPI processes.
5. What new challenges can arise.

11. AI/HPC Case Study

Select one application where both OpenMP and MPI can be considered.

Suggested applications:

- Image processing
- Deep learning data preprocessing
- Weather forecasting
- Computational fluid dynamics
- Medical image processing
- Scientific simulations
- Large-scale matrix computations
- Computer vision

Write **300–500 words** covering:

1. Nature of the computational problem.
2. Amount of data involved.
3. Whether shared or distributed memory is more appropriate.
4. Possible OpenMP implementation.

5. Possible MPI implementation.
6. Communication requirements.
7. Scalability considerations.
8. Whether a hybrid MPI + OpenMP approach could be useful.

12. Reflection Questions

1. Why can the same algorithm be implemented using both OpenMP and MPI?
2. What is the fundamental difference between OpenMP threads and MPI processes?
3. Which model requires explicit communication?
4. Why does MPI require data distribution?
5. Why does OpenMP generally have lower communication overhead on a single shared-memory machine?
6. Why can MPI scale to multiple nodes?
7. Why is matrix multiplication a good candidate for both OpenMP and MPI?
8. Why does 2D convolution require special consideration for boundary data in MPI?
9. What is a halo or ghost region?
10. Why might increasing threads/processes eventually stop improving performance?
11. How does problem size affect parallel speedup?
12. What factors should be considered before selecting OpenMP or MPI for an HPC application?

Answer the following based on your experimental observations:

1. Which programming model was easier to implement and why?
2. What was the major performance difference between OpenMP and MPI?
3. How did the problem size affect the comparison?
4. What communication overhead did you observe in MPI?
5. How did increasing the number of threads/processes affect performance?
6. Did the fastest implementation remain the same for both applications?
7. What did this experiment teach you about selecting a parallel programming model?
8. In a real HPC cluster, would you use OpenMP, MPI, or a combination? Explain your reasoning based on the architecture and workload.

14. Deliverables

Submit:

Source Code

1. Sequential Matrix–Matrix Multiplication
2. OpenMP Matrix–Matrix Multiplication
3. MPI Matrix–Matrix Multiplication
4. Sequential 2D Convolution
5. OpenMP 2D Convolution
6. MPI 2D Convolution

Performance Report

Include:

- Experimental setup
- CPU and system information
- Matrix/input sizes
- Number of threads/processes
- Execution times
- Speedup
- Efficiency
- Graphs
- OpenMP vs MPI comparison
- Architecture analysis
- AI/HPC case study
- Discussion-question answers
- Reflection
- Conclusion

15. Expected Learning Outcomes

After completing this assignment, students should be able to:

1. **Implement** the same computational problem using OpenMP and MPI.
2. **Apply** shared-memory and distributed-memory programming models.
3. **Measure** execution time, speedup and efficiency.
4. **Analyze** the scalability and communication overhead of OpenMP and MPI.
5. **Compare** different parallel programming models based on workload and architecture.
6. **Evaluate** the suitability of OpenMP, MPI and hybrid approaches for HPC applications.
7. **Relate** parallel programming techniques to real-world AI and HPC workloads.